Asistente de recomendaciÃ³n de pelÃculas mediante recuperaciÃ³n de informaciÃ³n y procesamiento de lenguaje natural

Danna Andrea Rodriguez Galarcio
IngenierÃa de Sistemas, Facultad de IngenierÃa
Universidad de San Buenaventura
Cali, Colombia
correo electrÃ³nico

***Abstract*—**

***Index Terms*—clasificaciÃ³n de texto, TF-IDF, modelos de lenguaje, embeddings multilingÃ¼es, atenciÃ³n al cliente, CRISP-ML(Q)**

I. IntroducciÃ³n

En los ultimos aÃ±os se ha visto un aumento en el acceso a peliculas a traves de plataformas de streaming, estas plataformas cuentan con un gran numero de usuarios los cuales en muchas ocasiones tienen la necesidad de poder contar con una herramienta que les permita encontrar contenido que se ajuste a sus intereses.

En este proyecto se propone el desarollo de un asistente virtual para la recomendacion de peliculas el cual permitira a los usuarios realizar sus preguntas en espaÃ±ol sobre las peliculas del catalogo y recibir informacion sobre las mismas, teniendo en cuenta caracteristias como titulo, genero, sinopsis, entre otros.

Es en este contexto que el poder implementar un sistema de clasificaciÃ³n automatica de texto resulta realmente util, el procesamiento PLN (Procesamiento del Lenguaje Natural) permite desarrollar sistemas capaces de procesar y analizar textos dados por el usuario y relacionarlos directamente con la informacion disponible, en este caso, informacion sobre peliculas.

Existen diferentes tecnicas que pueden considerarse para poder implementar un sistema con esas caracteristicas, en este caso, el sistema propone el uso de dos enfoques:

El primer enfoque consiste en la implementacion deTF-IDF (Term Frequencyâ€“Inverse Document Frequency) y similitud del coseno, el cual permite poder medir cual es la similutd del texto dado por el usuario en comparacion con la informacion disponible en el catalogo de peliculas,para de esta manera poder determinar cual es la pelicula que mas se ajusta a la pregunta realizada por el usuario mediante la coincidencia de las palabras, por otro lado.

El segundo enfoque consiste en usar un modelo preentrenado de embeddings, este modelo permite poder representar el texto dado por el usuario y la informacion disponible en el catalogo de peliculas en un espacio vectorial, este metodo tambien usa la similitud del coseno para pode medir la similitud entre los textos y de esta manera, poder identificar las peliculas que mas se relacionen a lo que pidio el usuario. Adicionalmente, cuenta con la incorporacion de Qween, un modelo de lenguaje LLM (Large Language Model) el cual permite poder generar embeddings de manera mas eficiente y precisa, este se incluyo ya que permite poder redactar una respuesta mas completa apartir de las peliculas encontradas.

La evaluacion se centrara en la comparacion entre estos dos enfoques para asi poder determinar la capacidad de cada enfoque para poder recuperar peliculas relevantes a partir de la pregunta realizada por el usuario, ademas de poder determinar cual de los dos enfoques es mas eficiente en cuanto a tiempo de respuesta y consumo de recursos. Las metricas de evaluacion implementadas consisten en Top-1, Recall@5, rango recÃproco medio (MRR) y tiempo de respuesta.

A. Objetivos

Desarrollar un asistente conversacional que permita a los usuarios realizar preguntas y recibir respuestas en espaÃ±ol para la recomendacion de peliculas mediante la implementacion y comparacion de dos metodos de recuperacion de informacion, uno basado en TF-IDF y otro con embeddings multilingÃ¼es, ambos con similitud del coseno.

Los objetivos especÃficos son:

1) Preparar un conjunto de datos de pelÃculas que incluya informaciÃ³n textual relevante, como tÃtulos, gÃ©neros y sinopsis, para utilizarlo en la recuperaciÃ³n de contenidos a partir de consultas en espaÃ±ol.
2) Implementar un mÃ©todo de recuperaciÃ³n basado en TF-IDF y similitud del coseno que permita comparar las consultas de los usuarios con la informaciÃ³n textual de las pelÃculas y recuperar los resultados mÃ¡s relacionados.
3) Implementar un mÃ©todo de recuperaciÃ³n semÃ¡ntica mediante embeddings multilingÃ¼es y similitud del coseno, con el fin de identificar pelÃculas relacionadas con la intenciÃ³n de las consultas, incluso cuando existan diferencias entre los tÃ©rminos utilizados por el usuario y los textos del catÃ¡logo.

4) Evaluar y comparar los dos mÃ©todos de recuperaciÃ³n mediante las mÃ©tricas Top-1, Recall@5, rango recÃproco medio (MRR) y tiempo de respuesta, utilizando un conjunto de consultas de prueba y criterios de evaluaciÃ³n definidos previamente.

B. JustificaciÃ³n

Para los usuarios, el realizar una busqueda de peliculas puede resultar una tarea tediosa e incluso compleja cuando los usuarios no conocen el titulo de la pelicula o en general, no saben que pelicula quieren ver y simplemente necesitan recomendaciones de un genero en especifico. Para este tipo de consultas, es importante considerar tecnicas de procesamiento de lenguaje natural que permiten poder relacionar las necesidades expresadas en las preguntas del usuario con las caracteristicas de las peliculas en el catalogo.

Este proyecto se justifica porque permite comparar dos enfoques distintos para poder cumplir con el proposito del proyecto, TF-IDF y embeddings multilingues, ambos combinados con la similitud del coseno. Esta combinacion permite determinar si las diferencias en los modelos se reflejan en el desempeÃ±o del sistema. La eleccion de las tecnicas tambien se justifica ya que permite construir un prototipo funcional con herramientas de acceso libre y gratuito sin depender de servicios que requieran de una suscripcion o pago.

Por otro lado, la incorporacion de un LLM permite transformar los resultados obtenidos en una respuesta mas completa y coherente para el usuario, esta capa se enfoca en la facilitacion de la presentacion de la informacion en pantalla, su funcionalidad se centra en unicamente redactar las respuestas a partir de la informacion obtenida en la busqueda, para de esta manera evitar el riesgo de que el modelo genere afirmaciones que no se encuentren respaldadas.

En general, el valor academico de este proyecto radica en que integra tecnicas de recuperacion de informacion y procesamiento de lenguaje, junto con sus metricas de evaluacion y el uso de Telegram como plataforma de interaccion con el usuario. La obtencion de resultados permite identificar las fortalezas y debilidades de cada enfoque sin asumir que una tecnica sera mejor que le otra.

De esta manera, al combinar todos estos factores se puede construir un prototipo funcional que permita a los usuarios realizar preguntas y recibir respuestas en espaÃ±ol sobre peliculas.

II. Estado del arte

A. RecuperaciÃ³n de informaciÃ³n textual

B. MÃ©todos tradicionales de recuperaciÃ³n: TF-IDF y similitud del coseno

C. Modelos preentrenados y embeddings semÃ¡nticos multilingÃ¼es

D. Sistemas de recomendaciÃ³n conversacional de pelÃculas

E. Modelos de lenguaje grandes para la generaciÃ³n de respuestas

F. MÃ©tricas y evaluaciÃ³n de sistemas de recuperaciÃ³n

G. AnÃ¡lisis comparativo y brecha identificada

III. MetodologÃa

A. Fase 1: ComprensiÃ³n de los negocios y los datos

Muchos usuarios pierden mas tiempo buscando que pelicula ver o intentando recordar su nombre que viendola. Para solucionar este problema, se desarrolla CineBot, un asistente que recibe una descripcion de una pelicula en espaÃ±ol y recomienda las peliculas que mas se parezcan a dicha descripcion de su catalogo, utilizando telegram como plataforma de comunicaciÃ³n.

1) Objetivo y criterio de Ã©xito: El objetivo de esta fase es mediante la creacion del sistema, poder comparar el uso de TF-IDF, un enfoque mas tradicional, con el modelo preentrenado embeddings multilingues, para de esta manera, ver que resultados arrojan ambos enfoques a la hora de realizar la misma tarea. Para esto, una vez que el usuario haga una consulta, el sistema evalua todo el catalogo y devuleve un listado ordenado por nivel de similitud, en el que en el primer lugar se encuentra la pelicula mas similar. En este caso, el criterio de exito es la posicion en la que ambos metodos colocan la pelicula correcta, las metricas que se tienen en cuenta son Top-1, Recall@5 y MRR, junto con un tiempo de respuesta de menos de un segundo por consulta.

2) Datos: Los datos usados para el sistema provienen del dataset *TMDB 5000 Movie Dataset* [**?**], este se eligio porque contiene informacion como titulo, sinopsis, genero, palabras clave, y otros atributos de aproximadamente 4,800 peliculas. De estas 4,800, se descartan aquellas que no tienen sinopsis, en este caso, se descartan 6 peliculas, quedando un total de 4,797.

En cuanto a la calidad de los datos, a pesar de ser un dataset que cuenta con toda la informacion necesaria sobre las peliculas para que el sistema pueda tener un catalogo solido, el que este solo cuente con 4,800 peliculas hace que sea considerado un dataset pequeÃ±o, por lo que contar con un conjunto de datos tan limitado limita la variedad de resultados que el bot podria ofrecer si contara con un catalogo mas grande. Por otro lado, la informacion esta completamente en ingles, por lo que aunque el usuario puede hacer sus consultas en espaÃ±ol, el bot solo puede responderle en ingles, por lo que la experiencia del usuario se ve limitada por la existencia de esta barrera del idioma. Por ultimo, la tercera limitacion con la que cuenta el dataset escogido es que este se trata de

TABLE I
COMPARACIÃ³N DE LOS TRABAJOS REVISADOS.

Referencia	AÃ±o	TÃ©cnica	Aporte principal	LimitaciÃ³n frente a este trabajo
[?]	1998	SVM lineal sobre caracterÃ-sticas TF-IDF dispersas	Muestra que los SVM lineales son adecuados para texto de alta dimensiÃ³n	Depende del vocabulario visto; no captura sinÃ³nimos ni otros idiomas
[?]	2019	Transformer bidireccional preentrenado (BERT)	Representaciones contextuales reutilizables en muchas tareas de PLN	La versiÃ³n original es en inglÃ©s; requiere ajuste fino con datos etiquetados
[?]	2019	Sentence-BERT (red siamesa)	Vectores de oraciones comparables con coseno, Ãºtiles para bÃºsqueda y agrupamiento	Los modelos originales se entrenan principalmente en inglÃ©s
[?]	2020	DestilaciÃ³n multilingÃ¼e de embeddings	Alinea el espacio vectorial entre idiomas a partir de un modelo maestro monolingÃ¼e	La calidad puede variar segÃºn el idioma y el dominio
[?]	2020	XLM-R, modelo enmascarado multilingÃ¼e	Transferencia entre idiomas con un Ãºnico modelo	Mayor tamaÃ±o y costo computacional
[?]	2019	ClasificaciÃ³n *zero-shot* mediante inferencia textual	Marco y bancos de prueba para clasificar sin ejemplos de las clases	Sensible a la formulaciÃ³n de las etiquetas
[?]	2020	Modelo de lenguaje de gran tamaÃ±o con aprendizaje en contexto	Resuelve tareas con instrucciones o pocos ejemplos, sin ajustar pesos	Modelo propietario y de gran tamaÃ±o; salida difÃcil de controlar
[?]	2024	Familia Qwen2.5 de modelos de lenguaje abiertos	Modelos abiertos que siguen instrucciones, con versiones pequeÃ±as	Salida textual libre: exige anÃ¡lisis y no entrega probabilidades
[?]	2024	RevisiÃ³n de literatura y AutoML	ClasificaciÃ³n de tickets de soporte con AutoML	completar tras leer
[?]	2021	Conjunto de clasificadores	ClasificaciÃ³n de tickets no estructurados de mesa de ayuda de TI	completar tras leer
Este trabajo	2026	TF-IDF, embeddings multilingÃ¼es y LLM *zero-shot*	ComparaciÃ³n de tres enfoques gratuitos con cambio de idioma entre entrenamiento y prueba	Entrenamiento sintÃ©tico; conjunto de prueba en espaÃ±ol pequeÃ±o

una base de datos de Kaggle totalmente estatica, por lo que no hay actualizaciones con nuevas peliculas y el bot no puede recomendar ninguna que halla sido estrenada despues de la fecha de creacion del dataset (27 de septiembre de 2017).

B. Fase 2: PreparaciÃ³n de datos

1) Limpieza y transformaciÃ³n de los datos: En el archivo original, los generos mas que decir simplemente el nombre del genero de la pelicula venian en un formato que incluia informacion adicional como un id, para solucionar este problema, se borra toda esta informacion adicional para consevar solamente el nombre del genero, por ejemplo, "action". De manera similar tambien se corta la fecha de estreno para conservar unicamente el aÃ±o en ves del dia, mes y aÃ±o de estreno de la pelicula, se extraen las palabras clave y se hace lo mismo para los esloganes.

```
def nombres(celda):
    try:
        return [d['name'] for d in json.loads(celda)]
    except Exception:
        return []

peliculas = crudo.copy()
peliculas['generos'] = peliculas['genres'].map(nombres)
peliculas['palabras_clave'] =
     peliculas['keywords'].map(nombres)
peliculas['anio'] =
     peliculas['release_date'].astype(str).str[:4]
peliculas['overview'] =
     peliculas['overview'].fillna('').str.strip()
peliculas['tagline'] =
     peliculas['tagline'].fillna('').str.strip()
```

Codigo 1. Procesamiento y estructuracion del texto de busqueda.

Otro filtro implementado consiste en filtrar las descripciones que no sean utiles para el sistema, ya que hay algunas peliculas que en su descripcion solo cuentan con frases cortas o poco informativas para el bot como "A great movie".

Por ultimo, tambien se eliminaron los posibles duplicados, ya que si una pelicula aparece dos veces en el dataset podria ocasionar que el sistema termine recomendando la misma pelicula dos veces.

```
n0 = len(peliculas)
peliculas = (peliculas[peliculas['overview'].str.len() >
     30]
          .drop_duplicates(subset='id').reset_index(drop=True))
print(f'Peliculas con sinopsis utilizable:
     {len(peliculas)} (descartadas: {n0 -
     len(peliculas)})')
```

Codigo 2.

2) ConstrucciÃ³n del texto de bÃºsqueda: La funcion texto_busqueda() se encarga de construir un texto que combine los generos y las sinopsis, ese texto combinado es el que usa el sistema a la hora de

En vez de que el sistema tenga que buscar directamente sobre todos los campos de una pelicula, se construye un texto que combina sus generos con la sinopsis y dicho texto es el que posteriormente usan los metodos de recomendacion TF-IDF y embeddings multilingues.

La funcion ficha se encarga de reunir los campos de una pelicula como titulo, aÃ±o, generos, puntuacion y un resumen de la sinopsis recortandola si supera los 180 caracteres, esto para que el bot pueda mostrarle al usuario una ficha con la informacion de la pelicula que le esta recomendando, por lo

que este texto no es el que se usa para calcular la similitud entre peliculas.

Por otro lado, se construye la variable CORPUS, *peliculas.to_dict('records')* se usa para transformar las filas del dataframe en una lista de diccionarios por pelicula. El sistema puede compara el primer elemento de cada tupla de CORPUS, que es el texto de busqueda, con la consulta del usuario y devuelve el segundo elemento de la tupla, que es la ficha de la pelicula recomendada, estos dos elementos son almacenados en CORPUS de la manera *(texto_de_busqueda, ficha_de_presentacion)*.

```
def texto_busqueda(f):
    return ', '.join(f['generos']) + '. ' + f['overview']

def ficha(f):
    gen = ', '.join(f['generos']) or 'sin genero'
    sinopsis = f['overview']
    if len(sinopsis) > 180:
        sinopsis = sinopsis[:180].rsplit(' ', 1)[0] + '?'
    return f"? {f['title']} ({f['anio']}) ? {gen} ? ?
        {f['vote_average']}\n{sinopsis}"

registros = peliculas.to_dict('records')
CORPUS = [(texto_busqueda(f), ficha(f)) for f in registros]

print(f'{len(CORPUS)} peliculas cargadas\n')
print(CORPUS[0][1])
```

Codigo 3.

Conjuntos de consulta. Los conjuntos de consulta se utilizan para probas el sistema y ver que tan bien funciona. El sistema cuenta con 3 conjuntos: A, B y C, y cada uno representa una manera diferente en la que se le puede preguntar al sistema por una pelicula:

- Conjunto A (usando el eslogan de la pelicula (tagline)): En este conjunto, la consulta se hace tomando el eslogan de una pelicula del dataset y utilizandola como si esa fuera la consulta que hizo el usuario.
- Conjunto B (usando las cinco primeras palabras clave): En el conjunto B, se toman las 5 primeras palabras claves que esten asociadas a una pelicula y estas se utilizan como si fueran la consulta.
- Conjunto C (usando las descripciones escritas en espaÃ±ol): Por ultimo, en el conjunto C en ves de usar un campo del dataset, se redacta una descripcion de una pelicula en espaÃ±ol y el sistema la relaciona con su titulo.

El uso de estos tres conjuntos resulta util ya que permite probar el comportamiento del sistema en 3 diferentes escenarios:

```
# ?? Conjunto A: eslogan
    ??????????????????????????????????????????????????????
cand = peliculas[peliculas['tagline'].str.len() > 10]
m = cand.sample(min(N_EVAL, len(cand)), random_state=42)
CONJUNTOS = {'A. Eslogan': (m['tagline'].tolist(),
    m['pos'].to_numpy())}

# ?? Conjunto B: palabras clave (las 5 primeras de TMDB)
    ??????????????????????
cand = peliculas[peliculas['palabras_clave'].map(len) >= 3]
m = cand.sample(min(N_EVAL, len(cand)), random_state=42)
CONJUNTOS['B. Palabras clave'] = ([' '.join(k[:5]) for k in
    m['palabras_clave']], m['pos'].to_numpy())

# ?? Conjunto C: descripciones en espanol escritas a mano
CONSULTAS_ES = [
    ('Un barco gigante choca con un iceberg y una chica rica
        se enamora de un chico pobre', 'Titanic'),
    #...mas descripciones escritas a mano...
]
```

Codigo 4.

Una vez hecho esto, se comprueban que los titulos existan en el catalogo, para eso, se recorre las descripciones y sus titulos, si se encuentra la pelicula en el catalogo, se guarda la descripcion y la posicion de la pelicula correcta, de no encontrarla, se guarda el titulo de la pelicula que no se encontro en faltan y se omite esa consulta

```
pos_por_titulo = (peliculas.sort_values('vote_count',
    ascending=False)
            .drop_duplicates('norm').set_index('norm')['pos'])

es_q, es_t, faltan = [], [], []

for q, titulo in CONSULTAS_ES:
    n = normalizar_titulo(titulo)
    if n in pos_por_titulo.index:
        es_q.append(q)
        es_t.append(int(pos_por_titulo[n]))
    else:
        faltan.append(titulo)

CONJUNTOS['C. Espanol (manual)'] = (es_q, np.array(es_t))
print(f'Conjunto C: {len(es_q)} consultas usables. No estan
    en el catalogo (omitidas): {faltan}\n')
```

Codigo 5.

Finalmente, la evaluacion de los conjuntos de consulta se hace recorriendo los conjuntos A,B y C, y aplica tanto TF-IDF como embeddings a las consultas de cada conjunto antes de calcular las metricas de evaluacion ya escogidas Top-1, Recall@5 y Recall@10 y MRR.

```
filas, PUNTAJES, POSICIONES = [], {}, {}
for conjunto, (consultas, objetivos) in CONJUNTOS.items():
    for metodo, fn in METODOS.items():
        S = fn(consultas)
        pos = posiciones(S, objetivos)
        PUNTAJES[(conjunto, metodo)], POSICIONES[(conjunto,
            metodo)] = S, pos
        filas.append({'Conjunto': conjunto, 'Metodo': metodo,
            'n': len(consultas),
                'Top-1': (pos <= 1).mean(), 'Recall@5':
                    (pos <= 5).mean(),
                'Recall@10': (pos <= 10).mean(), 'MRR': (1
                    / pos).mean()})

resultados = pd.DataFrame(filas)
```

Codigo 6.

C. Fase 3: IngenierÃa de modelos

Para el desarrollo del sistema, se implementaron dos enfoques con dos modelos de recuperacion de informacion diferentes, ambos utilizados bajo las mismas condiciones, es decir, con el mismo catalogo de peliculas. Los modelos se implementaron de la siguiente manera:

1) Enfoque 1: TF-IDF: La implementacion de este modelo se hizo siguiendo una serie de pasos, en el que despues de que se importan las librerias necesarias como numpy, counter y re, se definen las palabras vacias o *stopwords*, estas consisten en todas aquellas palabras que aunque son frecuentes, no aportan nada de informacion relevante al sistema, por ejemplo,

palabras como *el, la, de, que, etc.*. Esto se hace porque asi el sistema puede centrarse unicamente en las palabras que realmente ayuden a identificar el contenido de la pelicula.

Una vez definidas las palabras vacias, se limpia el texto, con limpiar se hace referencia a: convertir las palabras en minusculas, para que el sistema no trate las diferencias entre mayusculas y minusculas como palabras distintas, eliminar caracteres que no esten permitidos, como los signos de puntuacion y numeros los cuales son reemplazados por espacios. Por ultimo, se separan y filtran las palabras utilizando slip() para dividir el texto en palabras y eliminar aquellas que esten en la lista de STOPWORDS o que solo cuenten con un caracter.

```
def limpiar(texto):
    texto = texto.lower() # todo a minusculas
    texto = re.sub(r'[^a-zaeiouun ]', ' ', texto) # quitar
        puntuacion
    tokens = texto.split() # separar en palabras
    return [t for t in tokens if t not in STOP and len(t) >
        1] # filtrar stopwords
```

Codigo 7.

El siguiente paso es construir el indice TF-IDF, es decir, la estructura de datos que se construye para que el sistema pueda comparar la consulta que hace el usuario con las peliculas que tiene en su catalogo y asi encontrar la que mas se parezca. Si el sistema tiene una cantidad x de peliculas, el codigo procesa los textos de cada una de ellas y a partir de eso construye el vocabulario indentificando las palabras que aparecen en el catalogo, utilizando *word2idx* para asignarle a cada palabra una posicion numerica mientras que la variable *v* representa el numero total de palabras unicas del vocabulario. De esta manera, se puede representar a cada pelicula mediante un vector que tenga el mismo numero de posiciones que de palabras en el vocabulario.

```
def construir_indice(documentos):
    tokenizados = [limpiar(d) for d in documentos]
    N = len(tokenizados)

    # Vocabulario: todas las palabras unicas del corpus
    vocab = sorted({t for doc in tokenizados for t in doc})
    word2idx = {w: i for i, w in enumerate(vocab)}
    V = len(vocab)
```

Codigo 8.

La siguiente etapa es calcular los pesos TF-IDF asignandole un valor a cada palabra segun su frecuencia en cada pelicula y que tan comun es en el catalogo. Para poder lograr esto, lo primero que se hace es calcular el componente IDF (Inverse Document Frequency). La formula implementada es:

$$\text{idf}(t) = \ln\left(\frac{N+1}{\text{df}(t)+1}\right) + 1$$

Se le suma uno al numerador y al denominador para evitar la division por cero, ya que sin sumar +1, el codigo evaluaria una palabra que no aparece en ninguna pelicula del catalogo, df=0, por lo que la formula seria N/0, lo que generaria un error matematico. Tambien se le suma +1 al resultado final porque sin esa suma, si una palabra aprece en todas las peliculas del catalogo (df=N), el resultado de la formula seria 0, lo que no es deseable porque haria que esa palabra fuese ignorada por el sistema en la busqueda.

En el bucle se usa *set(doc)*para contar cada palabra una sola vez por pelicula al calcular df.

```
    # IDF de cada palabra
    df = np.zeros(V)
    for doc in tokenizados:
        for w in set(doc):
            if w in word2idx:
                df[word2idx[w]] += 1
    idf = np.log((N + 1) / (df + 1)) + 1 # +1 evita division
        por cero
```

Codigo 9.

Por ultimo, se calcula TF (Term Frequency), este concepto hace referencia a la frecuencia de cada palabra en cada pelicula, la formula implementada es:

$$\text{tf}(t,d) = \frac{\text{count}(t,d)}{|d|}$$

Se multiplica el resultado de TF por IDF, quedando de la forma:

$$\text{tf-idf}(t,d) = \text{tf}(t,d) \cdot \text{idf}(t)$$

Una palabra que aparece muchas veces en una pelicula, es decir, con una frecuencia alta, pero que al mismo tiempo, no esta presente en demasiadas peliculas recibe un peso mayor.

Posteriormente, se crea la matriz TF-IDF en la que representa cada pelicula mediante un vector de pesos TF-IDF. En el codigo, este paso se implemento como:

```
    # Matriz TF-IDF: una fila por pelicula, una columna por
        palabra
    matriz = np.zeros((N, V), dtype=np.float32) # float32:
        mitad de memoria
    for i, doc in enumerate(tokenizados):
        cnt = Counter(doc)
        total = len(doc) or 1
        for w, c in cnt.items():
            if w in word2idx:
                matriz[i, word2idx[w]] = (c / total) *
                    idf[word2idx[w]]
```

Codigo 10.

La normalizacion de los vectores se hace para que cada vector de pelicula tenga una longitud de 1, es decir, que todos los vectores esten en la misma escala. Esto se hace para ajustar la escala de los vectores y que asi, la comparacion se centre en la distribucion de los pesos en lugar de depender solo de la magnitud. La matriz resultante se guarda como el indice TF-IDF que utilizara el bot.

```
    # Normalizar cada fila (L2) para que la comparacion sea
        por direccion, no magnitud
    normas = np.linalg.norm(matriz, axis=1, keepdims=True)
    normas[normas == 0] = 1
    matriz /= normas

    return matriz, idf, word2idx
```

Codigo 11.

La funcion *vectorizar* se encarga de convertir una consulta de un usuario en un vector, para eso, aplica el mismo preprocesamiento y los mismos pesos IDF que se calcularon con

el catalogo de peliculas. Una vez hecho esto, se le asigna un peso a cada palabra conocida, teniendo en cuenta que solo se consideran las palabras que existen en el vocabulario del catalogo, asi que si una consulta contiene palabras que no aparecen en dicho vocabulario, esas palabras no le aportan peso al vector. Por ultimo, se normaliza la consulta para que de esta manera, tanto las peliculas como las consultas utilicen el mismo espacio vectorial y se les pueda comparar entre si:

```
def vectorizar(texto, idf, word2idx):
    tokens = limpiar(texto)
    vec = np.zeros(len(word2idx), dtype=np.float32)
    cnt = Counter(tokens)
    total = len(tokens) or 1

    for w, c in cnt.items():
        if w in word2idx:
            vec[word2idx[w]] = (c / total) * idf[word2idx[w]]

    norma = np.linalg.norm(vec)
    if norma > 0:
        vec /= norma
    return vec
```

Codigo 12.

A continuacion, se construye el indice del catalogo, este se construye una vez al iniciar el motor, en lugar de volverse a calcular todos los vectores cada vez que llega una consulta.

textos contiene los textos utilizados para calcular los vectores, *fichas* conserva la informacion que se mostrara al usuario cuando el sistema le recomiende una pelicula, *matriz_tfidf* almacena los vectores normalizados de todas las peliculas, *idf* contiene los pesos de las peliculas y *word2idx* permite localizar cada palabra en el vocabulario.

```

textos = [par[0] for par in CORPUS]
fichas = [par[1] for par in CORPUS]

matriz_tfidf, idf, word2idx = construir_indice(textos)

print(f'Motor TF-IDF listo ? {len(textos)} peliculas,
     {len(word2idx)} palabras en vocabulario')
```

Codigo 13.

Para encontrar las tres peliculas mas similares se usa la funcion *buscar_tfidf*, esta se encarga de vectorizar la consulta convirtiendo el texto del usuario en un vector normalizado, luego *matriz_tfidf @ vec* calcula el producto que hay entre el vector de consulta y el de cada pelicula, al estar estos vectores normalizados, el resultado termina siendo la similitud del coseno:

$$\text{similitud} = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|\|\vec{d}\|}$$

al estar los vectores normalizados pasa a ser:

$$\text{similitud} = \vec{q} \cdot \vec{d}$$

Los resultados, es decir, las peliculas, se ordendan de mayor a menor con *np.argsort(-similitudes)*, y por ultimo, con *[:k]* se devuelven los indices de las tres peliculas con las puntuaciones mas altas junto con sus similitudes.

```

def buscar_tfidf(consulta, k=3):
    # Convertir la consulta del usuario en vector TF-IDF
    vec = vectorizar(consulta, idf, word2idx)

    # Similitud con todas las peliculas
    # (producto punto entre vectores normalizados =
        similitud del coseno)
    similitudes = matriz_tfidf @ vec

    # Indices de las k peliculas mas similares
    top = np.argsort(-similitudes)[:k]
    return top, similitudes[top]
```

Codigo 14.

El umbral de similitud se aplica para permitir que el bot no devuelva automaticamente una pelicula cuando la similitud maxima es demasiado baja, asi que si la puntuacion de la pelicula mas cercana es igual o superior a 0.05, se devuelve su ficha, pero si esta por debajo el bot muestra un mensaje indicando que no encontro peliculas similares.

El valor de 0.05 es solo una configuracion inicial la cual puede ajustarse conforme se vaya probando el sistema.

```

UMBRAL_TFIDF = 0.05

def responder_tfidf(consulta, umbral=UMBRAL_TFIDF):
    idxs, sims = buscar_tfidf(consulta)
    if sims[0] >= umbral:
        return fichas[idxs[0]]
    return 'No encontre peliculas parecidas. Intenta
        reformular tu consulta o escribe /ayuda.'
```

Codigo 15.

Por ultimo, se ejecutaron una pruebas las cuales permiten verificar el comportamiendo del motor TF-IDF con diferentes tipos de entradas entre las que se encuentra una consulta en ingles y en espaÃ±ol sobre el mismo tema (robots), otra en ingles sobre dinosaurios y un parque tenamico y una consulta que esta fuera del contexto de peliculas.:

```

print("Probando el motor TF-IDF:\n")
pruebas = [
    'a robot that falls in love', # en ingles: deberia
        funcionar
    'una pelicula de robots que se enamoran', # en espanol:
        TF-IDF casi no ve palabras en comun
    'dinosaurs escape from a theme park',
    'receta de empanadas', # fuera de tema: debe dar fallback
]
for p in pruebas:
    print(f' Usuario: {p}')
    print(f' Bot: {responder_tfidf(p)}\n')
```

Codigo 16.

2) Enfoque 2: Embeddings multilingÃ¼es: La diferencia entre este segundo enfoque con el primero es que como ya se explico, con TF-IDF se representa las peliculas mediante pesos que estan asociados a palabras y la busqueda depende principalmente de que existan palabras en comun, mientras que por otro lado, con Embeddings se representa las peliculas mediante vectores mas densos que permiten encontrar los textos relacionandonos por su significado, es decir, que aunque no compartan las mismas palabras, si el significado es similar, el sistema puede encontrar la pelicula correcta, incluso si se utilizan palabras que esten en otro idioma.

Para la implemantacion de embeddings multilingues, se usaron las librerias torch para ejecutar operaciones de aprendizaje profundo y sentence-transformes para cargar modelos preentrenados de embeddings. Con TF-IDF, se necesita calcular manualmente todas las frecuencias de las palabras como con TF-IDF mientras que con embeddings, el modelo ya esta preentrenado y solo se necesita cargarlo e implementarlo para que el sistema pueda convertir los textos en vectores.

Para eso, el primer paso es seleccionar el dispositivo de ejecucion, se comprueba si hay una GPU disponible, si es asi, se ejecuta el modelo en la GPU con CUDA, de lo contrario, se ejecuta en la CPU utilizando el procesador. Con esto lo que se busca es poder aprovechar la GPU cuando este disponible para acelerar los calculos, ya que los modelos de embeddings suelen ser mas pesados y requieren mas recursos de computo que TF-IDF.

```
import torch
from sentence_transformers import SentenceTransformer

dispositivo = 'cuda' if torch.cuda.is_available() else 'cpu'
print('Dispositivo:', dispositivo)

emb_model =
    SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2',
    device=dispositivo)
```

Codigo 17.

Una vez hecho esto, se puede cargar el modelo preentrenado, *paraphrase* hace referencia a que como ya se comento antes, el modelo puede generar representaciones de frases para poder comparar su significado, *multilingual* permite trabajar con multiples idiomas y *MiniLM* hace referencia a la arquitectura del modelo en la que se procura que sea mas ligera que otros modelos mas grandes, mientras que el numero 12 hace referencia al numero de capas.

Este modelo recibe un texto y genera un vector numerico que representa el significado del mismo, asi si dos frases usan palabras diferentes pero tienen el mismo significado, el modelo puede representarlas mediante vectores cercanos, aun si las palabras estan escritas en idiomas diferentes.

```
emb_model =
    SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2',
    device=dispositivo)
```

Codigo 18.

El siguiente paso es convertir todas las peliculas en embeddings, con encode se convierte cada texto en un vector numerico, sus parametros utilizados son:

- textos: Consiste en una lista de textos de las peliculas que se convertiran en vectores
- batch_size: Se encarga de procesar los textos en grupos de hasta 64 para organizar el calculo.
- normalize_embeddings: Con este parametro se normalizan los vectores para que tengan una longitud igual a 1.
- show_progress_bar: Este parametro se usa para mostrar una barra de progreso mientras se generan los embeddings.

El resultado final se guarfa en la variable *matriz_emb*, esta es la matriz que contiene los emveddings de todas las peliculas, en este cada pelicula esta representada por cada fila de la matriz, mientras que cada columna representa una dimension del embedding y cada valor es un componente numerico del embedding. Las dimensiones son caracteristicas aprendidas por el modelo, mientras que los valores de las peliculas se calculan una sola vez y se conservan para las busquedas posteriores, al igual que con TF-IDF.

```
matriz_emb = emb_model.encode(textos, batch_size=64,
    normalize_embeddings=True, show_progress_bar=True)
```

Codigo 19.

La funcion *buscar_emb* recibe la consulta del usuario y se encarga de buscar las peliculas que tengan la mayor similitud con la consulta.

La consulta se convierte en un embedding, para esto, primero tiene que convertirse en un vector, asi, cuando el usuario hace una consulta el modelo genera un embedding que represente el significado de esa frase ingresada por el usuario.

Las similitudes se calculan mediante el producto entre la matriz de los embeddings de la consulta del usuario y la matriz de los embeddings de todas las peliculas, este resultado equivale a la similitud del coseno porque los vectores estan normalizados. Una consulta en espaÃ±ol puede hacer que el sistema le recomiende al usuario una pelicula con una sinopsis en ingles aunque por la diferncia de idioma, no halla ninguna coincidencia entre las palabras.

Las peliculas mas similares se seleccionan con *np.argsort(-similitudes)*, esta instruccion se encarga de ordenar las peliculas segun su similitud y selecciona las primeras k (3) peliculas con las puntuaciones mas altas.

Esta funcion termina retornando dos elementos, top, que contiene los indices de las tres peliculas seleccionadas, y similitudes[top], que contiene las puntuaciones de similitud de esas peliculas con la consultada por el usuario.

```
def buscar_emb(consulta, k=3):
    vec = emb_model.encode([consulta],
        normalize_embeddings=True)[0]
    similitudes = matriz_emb @ vec
    top = np.argsort(-similitudes)[:k]
    return top, similitudes[top]
```

Codigo 20.

El umbral de similitud se define estableciendo un valor inicial (0.30) para el motor de embeddings, este valor se utiliza posteriormente en el codigo en el comportamiento del bot.

Por otro lado, se comprueba que el motor esta listo para mostrar las dimensiones de la matriz de embeddings.

```
UMBRAL_EMB = 0.30

print(f'Motor de embeddings listo ? matriz
    {matriz_emb.shape}')
```

Codigo 21.

Para realizar las pruebas se hacen las mismas consultas de prueba que las que ya se usaron cuando se probo TF-IDF:

```
for p in ['a robot that falls in love', 'una pelicula de
     robots que se enamoran','dinosaurs escape from a theme
     park', 'receta de empanadas']:
    idxs, sims = buscar_emb(p)
    print(f' Usuario: {p}')
    print(f' Top-1: {peliculas.loc[idxs[0], "title"]}
         (similitud {sims[0]:.2f})\n')
```

Codigo 22.

a) LLM: Al enfoque 2 con embedding se le implemento esta capa extra que consiste en un modelo de lenguaje LLM, esto se incluyo principalmente porque aunque el sistema es funcional, ya que es capaz de encontrar las peliculas, el problema que tiene es que estos resultados que da se devuelven en forma de basica con una lista de peliculas con sus puntuaciones de similitud con lo que busco.

La ventaja que ofrece el LLM es que el sistema pueda tomar las peliculas encontradas y en base a ellas, generar una respuesta mas elaborada y natural para el usuario.

La implementacion en el codigo se llevo de la siguiente manera, en la que despues de importar y cargar el modelo Qween, se definen las instrucciones del asistente y se crea la funcion *asistente*. En esta, se recibe la consulta del usuario y se genera una respuesta mediante el LLM, despues de comprobar que el modelo esta cargado, el asistente usa la funcion *buscar_emb* para recuperar las 3 peliculas que se determinaron como las mas similares a lo que busca el usuario, luego, se construye el contexto que el cual contiene la informacion relevante sobre las peliculas como el titulo, aÃ±o, generos y los primeros 300 caracteres de la sinopsis.

Esto es importante porque Qween necesita poder recibir la informaciÃ³n sobre las peliculas para de esa manera, poder elaborar una recomendacion en base a los resultados que encontro el motor de embeddings sin tener que realizar una nueva busqueda en todo el catalogo.

En la variable de texto_chat, se convierten los mensajes al formato del modelo, ya que para funcionar, en este caso necesita seguir la plantilla de conversacion que utiliza Qween *apply_chat_template*. Posteriormente, se tokeniza la entrada transformando el texto en numeros que el modelo puede procesar y los resultados se devuelven como tensonres de pyTorch, que es la libreria que se utiliza para ejecutar el modelo, con *.to(dispositivo)* se colocan estos tensores en el dispositivo donde esta cargado el modelo.

La salida se genera con *llm_model.generate*, y se decodifica con *llm_tok.decode* para convertir los numeros de vuelta a texto, que es lo que finalmente se devuelve al usuario, el modelo devuelve una secuencia de tokens que contiene tanto la entrada original como los tokens generados.

```
def asistente(consulta, k=3):
    if llm_model is None:
        return 'El asistente con LLM no esta activado
             (USE_LLM = False).'
    idxs, sims = buscar_emb(consulta, k)
    contexto = '\n'.join(
        f"- {peliculas.loc[i, 'title']} ({peliculas.loc[i,
             'anio']}), generos: {', '.join(peliculas.loc[i,
             'generos'])}. "
        f"Sinopsis: {peliculas.loc[i, 'overview'][:300]}"
        for i in idxs)
    mensajes = [
        {'role': 'system', 'content': SYSTEM_LLM},
        {'role': 'user', 'content': f'Consulta del usuario:
             {consulta}\n\nPeliculas
             encontradas:\n{contexto}\n\n'
                                 'Recomienda las que mejor
                                      encajan y explica en una
                                      frase por que.'},
    ]
    texto_chat = llm_tok.apply_chat_template(mensajes,
         tokenize=False, add_generation_prompt=True)
    entrada = llm_tok(texto_chat,
         return_tensors='pt').to(dispositivo)
    with torch.no_grad():
        salida = llm_model.generate(**entrada,
             max_new_tokens=200, do_sample=False,
                                 pad_token_id=llm_tok.eos_token_id)
    return llm_tok.decode(salida[0,
         entrada['input_ids'].shape[1]:],
         skip_special_tokens=True).strip()
```

Codigo 23.

TABLE II
RESULTADOS EN EL CONJUNTO B (ESPAÃ±OL, $n = 60$).

MÃ©todo	Exact.	Prec.	Exh.	F1	ms/cons.
TF-IDF	–	–	–	–	–
Embeddings	–	–	–	–	–
LLM *zero-shot*	–	–	–	–	–

Por ultimo, se realizan pruebas para revisar el comportamiento de esta parte del sistema. En este caso, se usan las 3 primeras consultas del conjunto de descripciones en espaÃ±ol y una consulta adicional sobre las peliculas de terror con casas embrujadas. Tambien se mide cuanto tiempo se demora con cada respuesta usando time.time().

```
if USE_LLM:
    for q in es_q[:3] + ['una pelicula de terror con casas
         embrujadas']:
        t0 = time.time()
        r = asistente(q)
        print(f'Usuario: {q}\nCineBot: {r}\n({time.time() -
             t0:.1f} s)\n' + '-' * 60)
```

Codigo 24.

D. EvaluaciÃ³n del modelo

E. EvaluaciÃ³n y aseguramiento de la calidad

F. Despliegue e integraciÃ³n del asistente

IV. Resultados

A. Conjunto A: Bitext (inglÃ©s)

B. Conjunto B: espaÃ±ol (manual)

C. AnÃ¡lisis de errores

D. Tiempo de respuesta

V. DiscusiÃ³n

Brecha entre conjuntos.
Costo y beneficio.
AmbigÃ¼edad de las categorÃas.
Amenazas a la validez.
Trabajo futuro.

VI. Conclusiones

Escribir tres o cuatro conclusiones basadas en tus resultados, cada una respondiendo a un objetivo especÃfico: (1) quÃ© enfoque generaliza mejor al espaÃ±ol y por cuÃ¡nto; (2) cuÃ¡l es el costo en tiempo de cada enfoque; (3) quÃ© limitaciones tiene el estudio; (4) quÃ© se harÃa despuÃ©s.

El cÃ³digo del cuaderno, la interfaz y el bot se entregan junto con este artÃculo.

Agradecimientos